

Интеграция TLS-метаданных в систему анализа трафика

Техническое задание для backend-разработки

Обоснование выбора метрик и порогов

Почему JA3 + SNI. JA3 [1, 2] вычисляется из открытой части TLS-рукопожатия (ClientHello) и не требует расшифровки трафика. Метод поддержан в Suricata, Zeek, Cloudflare, Elastic и большинстве NDR-систем [3]. SNI добавляет к нему контекст назначения. Этой пары достаточно для всех трёх детекторов; JA3S, JARM и ML-классификаторы — избыточны для MVP.

Доля доминирующего JA3 вместо энтропии Шеннона. Энтропия [4] — классический инструмент детекции аномалий, но плохо интерпретируема: $H = 2.7$ ничего не говорит оператору, тогда как «92% потоков имеют один JA3» — говорит. Для нашего случая (поиск однородного ботнета) обе метрики эквивалентны, доля — проще.

Абсолютный порог + множитель роста вместо Z-score для перегрузки. Z-score требует оценённого baseline. Для офлайн-анализа рсар его неоткуда взять — baseline пришлось бы строить из самого файла, то есть сравнивать файл с собой. Простой порог честнее.

Конкретные значения порогов.

- `WormDominantJA3Ratio = 0.80` — червь даёт 0.95–1.00; запас оставлен на фоновые легитимные TLS-сессии [5].
- `WormMinFlows = 30` — защита от ложных срабатываний на малых выборках ($p_{\text{dom}} = 1.0$ при $N = 3$ — шум).
- `WormMinUniqueDst = 20` — браузер за минуту обращается к 5–15 уникальным IP, червь — к десяткам.
- `WormInternalDstRatio = 0.70` — червь распространяется в LAN (Conficker, WannaCry); порог отсекает обычный исходящий в Интернет.
- `DDoSMinFlows = 5000`, `DDoSDominantJA3Ratio = 0.90` — ботнет на единой C&C-инфраструктуре [1] даёт идентичный JA3 у $\geq 90\%$ ботов; 5000 потоков — порог отсеечения легитимных всплесков.

Все значения — стартовые, требуют калибровки на реальных рсар (раздел 8).

1 Подключение библиотеки

```
1 go get github.com/refraction-networking/utls
```

Добавить зависимость в `go.mod`. Библиотека `utls` позволяет разбирать `ClientHello` из сырых байтов без установки соединения и предоставляет готовую функцию для расчёта JA3.

2 Расширение структуры потока

Файл: `internal/packet/stats.go`

```
1 type TLSInfo struct {
2     JA3 string // отпечаток клиента TLS
3     SNI string // имя сервера из ClientHello
4 }
5
6 type FlowStats struct {
7     // ... существующие поля ...
8     TLS *TLSInfo // nil, если поток не TLS или ClientHello не
        получен
9 }
```

Поле `TLS` остаётся `nil` в подавляющем большинстве потоков, дополнительная память расходуется только на TLS-сессии.

3 Извлечение TLS-метаданных

Файл: `internal/packet/processor.go`

3.1 Условия запуска парсера

Парсинг `ClientHello` выполняется **только при одновременном выполнении** следующих условий:

1. это первый пакет TCP-потока с непустым `payload`;
2. порт назначения — 443 (TLS на нестандартных портах вне scope MVP);
3. `payload[0] == 0x16` (TLS record type: handshake);
4. `len(payload) >= 6` и `payload[5] == 0x01` (handshake type: `ClientHello`).

Проверка `handshake type` обязательна: иначе попадают `ServerHello` и другие записи, которые `utls` не сможет распарсить, — это лишние ошибки и лишняя нагрузка.

3.2 Реализация

```
1 import "github.com/refraction-networking/utls"
2
3 func extractTLS(payload []byte) *TLSInfo {
4     if len(payload) < 6 || payload[0] != 0x16 || payload[5] != 0x01 {
5         return nil
6     }
7     ch, err := utls.UnmarshalClientHello(payload)
8     if err != nil || ch == nil {
9         return nil
10    }
11    return &TLSInfo{
12        JA3: utls.JA3String(ch),
13        SNI: ch.ServerName,
14    }
15 }
```

Результат сохраняется в `FlowStats.TLS`. Парсер не должен паниковать на обрезанных или повреждённых пакетах — любая ошибка превращается в `nil`.

4 Использование в детекторах

4.1 Перегрузка (Overload)

Файл: `internal/detectors/overload_detector.go`

В существующее окно статистики добавить:

- счётчик потоков с `TLS != nil` в текущем окне (`ClientHelloCount`);
- счётчик в предыдущем окне (`ClientHelloCountPrev`).

Алерт срабатывает, если:

$$\text{ClientHelloCount} > \text{OverloadAbsoluteThreshold}$$

или

$$\frac{\text{ClientHelloCount}}{\max(\text{ClientHelloCountPrev}, 1)} > \text{OverloadGrowthFactor}$$

Обоснование: статистические методы (Z-score) требуют надёжной оценки μ и σ по историческому окну. Для офлайн-анализа рсар-файла `baseline` неоткуда взять, кроме как из самого анализируемого файла, — это даёт смещённую оценку. Простой абсолютный порог плюс множитель роста относительно предыдущего окна решает ту же задачу без скрытых допущений. Z-score рассматривается как возможное расширение (см. раздел 9).

4.2 Черви (Worm Detection)

Файл: `internal/detectors/worm_detector.go`

Для каждого `srcIP` собрать TLS-потoki в окне и вычислить:

- N — общее число TLS-потокoв от `srcIP`;
- доминирующий JA3 и его доля p_{dom} среди всех JA3 от этого `srcIP`;
- U — число уникальных `dstIP`;
- r_{int} — доля внутренних `dstIP` (по списку RFC1918 либо по конфигурируемому списку сетей).

Алерт срабатывает при выполнении всех условий:

1. $N \geq \text{WormMinFlows}$ (по умолчанию 30);
2. $p_{\text{dom}} \geq \text{WormDominantJA3Ratio}$ (по умолчанию 0.80);
3. $U \geq \text{WormMinUniqueDst}$ (по умолчанию 20);
4. $r_{\text{int}} \geq \text{WormInternalDstRatio}$ (по умолчанию 0.70).

Обоснование минимума потокoв: без условия $N \geq 30$ обычный браузер, открывший 3 вкладки, даст $p_{\text{dom}} = 1.0$ и попадет под подозрение. Минимальный объем выборки — защита от ложных срабатываний на легитимных малочисленных сессиях.

4.3 DDoS (ботнет)

Файл: `internal/detectors/ddos_detector.go`

Для каждого `dstIP` (потенциальной жертвы) в окне:

- M — число входящих TLS-потокoв на `dstIP`;
- доминирующий JA3 среди этих потокoв и его доля q_{dom} .

Алерт срабатывает при:

1. $M \geq \text{DDoSMinFlows}$ (по умолчанию 5000);
2. $q_{\text{dom}} \geq \text{DDoSDominantJA3Ratio}$ (по умолчанию 0.90).

Обоснование: ботнет, использующий единый TLS-стек, даёт характерную сигнатуру — подавляющее большинство `ClientHello` на жертву идентичны по JA3. Доля одного JA3 — более прямой и интерпретируемый сигнал, чем энтропия распределения. Энтропийный подход рассматривается как возможное расширение (см. раздел 9).

5 Конфигурация

Все пороги выносятся в конфигурационный файл (формат на усмотрение реализации, ожидаемо — YAML/JSON в той же схеме, что и существующие настройки детекторов).

Параметр	Тип	Значение по умолчанию
OverloadAbsoluteThreshold	int	1000 CH / окно
OverloadGrowthFactor	float	5.0
WormMinFlows	int	30
WormDominantJA3Ratio	float	0.80
WormMinUniqueDst	int	20
WormInternalDstRatio	float	0.70
WormInternalNetworks	[]CIDR	RFC1918 + RFC4193
DDoSMinFlows	int	5000
DDoSDominantJA3Ratio	float	0.90

Размер окна и механизм скользящего среднего — использовать существующие в проекте, новые структуры не вводить.

6 Формат вывода алертов

Каждое срабатывание детектора, использующего TLS-сигналы, должно расширять существующий формат алерта полями:

```
1 type TLSAlertContext struct {
2     DominantJA3    string    // доминирующий отпечаток JA3 в выборке
3     DominantRatio float64    // его доля, от 0.0 до 1.0
4     SampleSize     int       // объём выборки потоков
5     SampleSNI      []string  // до 5 представительных SNI для отчёта
6 }
```

Это позволит оператору видеть, по какому JA3 сработала эвристика, и быстро искать его в публичных базах (например, ja3er.com, abuse.ch SSL Blacklist).

7 Граничные случаи и ограничения

- **Фрагментированный ClientHello.** В редких случаях TLS-запись разбивается на несколько TCP-сегментов. В MVP такие потоки не парсятся (TLS = nil); это допустимая потеря, так как доля подобных сессий незначительна.
- **TLS на нестандартных портах.** В MVP не обрабатываются. Если потребуется — расширить условие порта в конфиге.
- **QUIC (UDP/443).** Полностью вне scope: JA3 для QUIC не определён, требуется JA3Q/JA4. Зафиксировать как ограничение.

- **ECH / Encrypted ClientHello.** SNI зашифрован, JA3 в общем виде по-прежнему вычислим. Доля ECH в наблюдаемом трафике пока мала, но будет расти — это известное ограничение метода.
- **Производительность.** Парсинг ClientHello вызывается только для первого пакета потока с ранним выходом по байтам сигнатуры. Накладные расходы — порядка одного вызова `utls.UnmarshalClientHello` на TLS-сессию, что приемлемо.

8 Тестирование

Минимальный набор фикстур для регрессии:

1. **clean_browsing.pcap** — обычный веб-сёрфинг (Chrome/Firefox), детекторы должны молчать.
2. **worm_synthetic.pcap** — синтетический трафик: один `srcIP` с одним JA3 на 50+ внутренних `dstIP`.
3. **ddos_botnet.pcap** — 5000+ потоков на одну жертву с одинаковым JA3.
4. **malformed_tls.pcap** — обрезанные/повреждённые TLS-записи; парсер не должен падать.

Источники реальных образцов червей и ботнет-трафика: `malware-traffic-analysis`, `stratosphereips.org` (CTU-13).

9 Дальнейшие улучшения (вне MVP)

- **JA4.** Современная замена JA3 (FoxIO), включает информацию о порядке расширений и ALPN. На момент задачи нативной поддержки в `utls` нет; реализуется отдельно.
- **Z-score для перегрузки.**

$$z = \frac{x - \mu}{\sigma}, \quad \text{алерт при } z > 3.0$$

Требует надёжного baseline, актуально для онлайн-режима, не для офлайн-pcap.

- **Энтропия Шеннона для DDoS.**

$$H = - \sum_i p_i \log_2 p_i$$

по распределению JA3 на `dstIP`, алерт при $H < H_{thr}$. Эквивалентно текущему подходу с долей доминирующего JA3, но менее интерпретируемо.

- **Сопоставление JA3 с публичными базами вредоносных отпечатков** (`abuse.ch`). Превращает эвристику в сигнатурный детектор.
- **Поддержка QUIC** через JA4Q.

Список литературы

- [1] J. B. Althouse, J. Atkinson, J. Atkins. *TLS Fingerprinting with JA3 and JA3S*. Salesforce Engineering Blog, 2019. <https://engineering.salesforce.com/tls-fingerprinting-with-ja3-and-ja3s-247362855967/>
- [2] Salesforce. *JA3 — A method for profiling SSL/TLS Clients*. GitHub, 2017. <https://github.com/salesforce/ja3>
- [3] Fastly Security Research Team. *The state of TLS fingerprinting: What's Working, What Isn't, and What's Next*. Fastly Blog, 2024. <https://www.fastly.com/blog/the-state-of-tls-fingerprinting-whats-working-what-isnt-and-whats-next>
- [4] A. Lakhina, M. Crovella, C. Diot. *Mining Anomalies Using Traffic Feature Distributions*. Proceedings of ACM SIGCOMM 2005, pp. 217–228.
- [5] H.-A. Kim, B. Karp. *Autograph: Toward Automated, Distributed Worm Signature Detection*. Proceedings of the 13th USENIX Security Symposium, 2004.
- [6] Refraction Networking. *uTLS — Fork of the Go standard TLS library, providing low-level access to the ClientHello*. GitHub. <https://github.com/refraction-networking/utls>
- [7] J. Althouse et al. *JA4+ Network Fingerprinting*. FoxIO, 2023. <https://github.com/FoxIO-LLC/ja4>
- [8] abuse.ch. *SSL Blacklist (SSLBL) — JA3 Fingerprint Blacklist*. <https://ssllbl.abuse.ch/ja3-fingerprints/>